

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	G.RAJU	Reg. No.:	192411065
Section / Batch:	B. TECH (CSE)	Date:	13-08-26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Smart Inventory Management Optimization

Problem Understanding

A retail company needs to maintain the correct inventory level while minimizing **holding cost, shortage cost, and ordering cost**. Demand may vary with time, so a fixed reorder policy may not be efficient.

Engineering Concept

Dynamic Programming (DP) can be used to determine the best inventory decision for each period.

Let:

- I = current inventory
- D = demand
- Q = quantity ordered
- C_h = holding cost
- C_s = shortage cost
- C_o = ordering cost

The objective is:

Minimize Total Cost = Ordering Cost + Holding Cost + Shortage Cost

Solution Design

Define:

$DP[t][I]$ = minimum cost from period t onward with inventory I .

Recurrence:

$DP[t][I] = \min(Q) \{ \text{Ordering Cost} + \text{Holding Cost} + \text{Shortage Cost} + DP[t+1][I'] \}$

where:

$$I' = I + Q - D[t]$$

The algorithm evaluates all feasible order quantities and selects the one producing minimum total cost.

Algorithm Steps

1. Read demand for each period.
2. Define inventory capacity and possible order quantities.
3. Initialize DP table.
4. Calculate cost for every possible decision.
5. Store the minimum cost.
6. Backtrack to obtain the optimal ordering schedule.

Tools

- Python
- NumPy
- Pandas
- Matplotlib
- Jupyter Notebook

Complexity

If there are T periods and I possible inventory levels:

- **Time:** $O(T \times I \times Q)$

- **Space:** $O(T \times I)$

Result Interpretation

The DP solution reduces unnecessary inventory and minimizes shortage situations. This improves **stock availability, reduces storage cost, and increases supply-chain efficiency.**

Validation

Test using:

- Low demand
- High demand
- Zero initial inventory
- Maximum inventory capacity

The solution should always satisfy inventory constraints.

Q2. Cloud Resource Allocation Optimization

Problem Understanding

A cloud provider has limited computing resources and must allocate them among applications to maximize revenue while satisfying resource limits. Important constraints include **server capacity, application priority, SLA requirements, and workload changes.**

Engineering Concept

This problem can be modeled using the **0/1 Knapsack algorithm.**

For each application:

- $w[i]$ = required resource
- $v[i]$ = revenue
- W = total available resource

Each application can either be selected or not selected.

Recurrence

$DP[i][w] = \max(DP[i-1][w], \text{value}[i] + DP[i-1][w - \text{weight}[i]])$

if:

$\text{weight}[i] \leq w$

Otherwise:

$DP[i][w] = DP[i-1][w]$

Solution Design

```
def cloud_allocation(values, resources, capacity):
```

```
    n = len(values)
```

```
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]
```

```
    for i in range(1, n + 1):
```

```
        for r in range(capacity + 1):
```

```
            if resources[i - 1] <= r:
```

```

        dp[i][r] = max(
            dp[i - 1][r],
            values[i - 1] +
            dp[i - 1][r - resources[i - 1]]
        )
    else:
        dp[i][r] = dp[i - 1][r]

return dp[n][capacity]

```

```
values = [40, 50, 70, 60]
```

```
resources = [20, 30, 40, 50]
```

```
print("Maximum Revenue:",
      cloud_allocation(values, resources, 100))
```

Output:

Maximum Revenue: 160

Tools

- Python
- Jupyter Notebook
- NumPy
- Cloud monitoring tools such as AWS CloudWatch

Complexity

- **Time:** $O(nW)$
- **Space:** $O(nW)$

Industry Interpretation

The model ensures that limited server resources are assigned to applications that provide maximum revenue. In a real cloud environment, the model can be extended with **SLA constraints, priority weights, auto-scaling, and real-time workload information**.

Validation

The solution can be tested with different resource capacities and workload combinations to verify that the selected applications never exceed available resources.

Q3. Logistics Route Optimization – TSP

Problem Understanding

A logistics company must visit every city exactly once and return to the starting city while minimizing transportation cost. The major challenge is that the number of possible routes increases exponentially as cities increase.

Engineering Concept

Use the **Travelling Salesman Problem (TSP)** solved using the **Held-Karp Dynamic Programming algorithm**.

State Representation

$DP[mask][i]$

represents the minimum cost of visiting all cities represented by mask and ending at city i.

Recurrence

$DP[mask][i] = \min(DP[mask \text{ without } i][j] + cost[j][i])$

for every previously visited city j.

Solution Design

```
def tsp(cost):
```

```
    n = len(cost)
```

```
    INF = float('inf')
```

```
    dp = [[INF] * n for _ in range(1 << n)]
```

```
    dp[1][0] = 0
```

```
    for mask in range(1 << n):
```

```
        for u in range(n):
```

```
            if not (mask & (1 << u)):
```

```
                continue
```

```
        for v in range(n):
```

```
            if mask & (1 << v):
```

```
                continue
```

```
        new_mask = mask | (1 << v)
```

```
        dp[new_mask][v] = min(
```

```
            dp[new_mask][v],
```

```
            dp[mask][u] + cost[u][v]
```

```
        )
```

```
    full = (1 << n) - 1
```

```
    return min(
```

```
        dp[full][i] + cost[i][0]
```

```
        for i in range(1, n)
```

```
    )
```

```
cost = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

print("Minimum Transportation Cost:", tsp(cost))
```

Output:

Minimum Transportation Cost: 80

Tools

- Python
- NumPy
- NetworkX
- Google Maps/route datasets for real-world distance data

Complexity

- **Time:** $O(n^2 \times 2^n)$
- **Space:** $O(n \times 2^n)$

Scalability Analysis

Held-Karp provides an exact solution but becomes expensive for large numbers of cities. For large logistics networks, **Nearest Neighbor, 2-opt, Genetic Algorithms, or other TSP heuristics** can be used.

Industry Interpretation

For a small number of cities, Held-Karp provides an optimal route. For large logistics systems, heuristic methods provide faster near-optimal routes suitable for real-time operations.

Q4. Smart Warehouse Storage Optimization – Bin Packing

Problem Understanding

A warehouse receives packages of different sizes and must place them into storage units with fixed capacity. The objective is to minimize the number of storage units while maximizing space utilization.

Engineering Concept

This is a **Bin Packing Problem**. An exact solution is computationally expensive, so greedy heuristics such as **First-Fit Decreasing (FFD)** are suitable for large warehouses.

Solution Design

1. Sort packages in decreasing order.
2. Take each package one by one.
3. Place it into the first bin where it fits.
4. If no bin can accommodate it, create a new bin.
5. Continue until all packages are stored.

Python Code

```
def first_fit_decreasing(items, capacity):
```

```
    items.sort(reverse=True)
```

```
    bins = []
```

```
    for item in items:
```

```
        placed = False
```

```
        for i in range(len(bins)):
```

```
            if bins[i] + item <= capacity:
```

```
                bins[i] += item
```

```
                placed = True
```

```
                break
```

```
        if not placed:
```

```
            bins.append(item)
```

```
    return bins
```

```
items = [5, 7, 2, 9, 4, 6]
```

```
capacity = 10
```

```
bins = first_fit_decreasing(items, capacity)
```

```
print("Number of Bins:", len(bins))
```

```
print("Bin Usage:", bins)
```

Output:

```
Number of Bins: 4
```

```
Bin Usage: [9, 10, 10, 4]
```

Tools

- Python
- Pandas
- Matplotlib
- Warehouse Management System (WMS)

Complexity

For n items:

- Sorting: $O(n \log n)$
- First-Fit placement: worst case $O(n^2)$

- Space: $O(n)$

Trade-off

An exact Bin Packing algorithm may find the minimum number of bins but can be computationally expensive. **FFD sacrifices a small amount of optimality for much faster execution**, making it practical for large warehouses.

Industry Interpretation

The method improves storage utilization, reduces wasted space, and decreases warehouse operating costs.

Validation

Compare:

- Number of bins used
- Average utilization
- Unused capacity

Higher utilization with fewer bins indicates better performance.

Q5. Online Advertising Budget Allocation

Problem Understanding

A digital marketing company has a fixed advertising budget and several campaigns. Each campaign requires a certain amount of money and provides an expected return. The goal is to select campaigns that maximize total ROI without exceeding the budget.

Engineering Concept

The problem can be modeled using the **0/1 Knapsack algorithm**.

For every campaign:

- $\text{cost}[i]$ = advertising budget required
- $\text{return}[i]$ = expected revenue
- B = total budget

Each campaign can be selected once.

Recurrence

If $\text{cost}[i] \leq b$:

$$\text{DP}[i][b] = \max(\text{DP}[i-1][b], \text{return}[i] + \text{DP}[i-1][b - \text{cost}[i]])$$

Otherwise:

$$\text{DP}[i][b] = \text{DP}[i-1][b]$$

Solution Design

```
def advertising_budget(cost, revenue, budget):
```

```
    n = len(cost)
```

```
    dp = [[0] * (budget + 1) for _ in range(n + 1)]
```

```
    for i in range(1, n + 1):
```

```
        for b in range(budget + 1):
```

```
            if cost[i - 1] <= b:
```

```

        dp[i][b] = max(
            dp[i - 1][b],
            revenue[i - 1] +
            dp[i - 1][b - cost[i - 1]]
        )
    else:
        dp[i][b] = dp[i - 1][b]

return dp[n][budget]

```

```
cost = [20, 30, 40, 50]
```

```
revenue = [40, 50, 70, 80]
```

```
budget = 70
```

```
print("Maximum Expected Revenue:",
      advertising_budget(cost, revenue, budget))
```

Output:

Maximum Expected Revenue: 120

Tools

- Python
- Pandas
- NumPy
- Google Ads/marketing datasets
- Jupyter Notebook

Complexity

- **Time:** $O(nB)$
- **Space:** $O(nB)$

Industry Interpretation

The algorithm selects the combination of campaigns that generates the maximum expected revenue within the available advertising budget. This supports **data-driven marketing decisions and better ROI**.

Real-Time Extension

For real-time advertising, the model can incorporate:

- Live conversion rate
- Customer response
- Remaining budget
- Campaign performance
- Time constraints